4

	8	

1

1

```
-u
0780:000E B87E07        MOV     AX,077E
0780:0011 8ED8          MOV     DS,AX
0780:0013 B435          MOV     AH,35
0780:0015 B01C          MOV     AL,1C
0780:0017 CD21          INT     21
0780:0019 06            PUSH    ES
0780:001A 58            POP     AX
0780:001B A30E00        MOV     [000E],AX
0780:001E 891E0C00      MOV     [000C],BX
0780:0022 0E            PUSH    CS
0780:0023 58            POP     AX
0780:0024 BA0000        MOV     DX,0000
0780:0027 8ED8          MOV     DS,AX
0780:0029 B425          MOV     AH,25
0780:002B B01C          MOV     AL,1C
0780:002D CD21          INT     21
```

```
0780:002F B87E07        MOV     AX,077E
0780:0032 8ED8          MOV     DS,AX
0780:0034 B401          MOV     AH,01
0780:0036 CD21          INT     21
0780:0038 3C51          CMP     AL,51
0780:003A 75F8          JNZ     0034
0780:003C 8B160C00      MOV     DX,[000C]
0780:0040 A10E00        MOV     AX,[000E]
0780:0043 8ED8          MOV     DS,AX
0780:0045 B425          MOV     AH,25
0780:0047 B01C          MOV     AL,1C
0780:0049 CD21          INT     21
0780:004B B87E07        MOV     AX,077E
0780:004E 8ED8          MOV     DS,AX
```

```
0780:0050 A10A00        MOV     AX,[000A]
0780:0053 B90400        MOV     CX,0004
0780:0056 8D3E1000      LEA     DI,[0010]
0780:005A 51            PUSH    CX
0780:005B B104          MOV     CL,04
0780:005D D3C0          ROL     AX,CL
0780:005F 59            POP     CX
0780:0060 8AD8          MOV     BL,AL
0780:0062 80E30F        AND     BL,0F
0780:0065 80C330        ADD     BL,30
0780:0068 80FB39        CMP     BL,39
0780:006B 7603          JBE     0070
0780:006D 80C307        ADD     BL,07
```

```
0780:0070 881D          MOV     [DI],BL
0780:0072 47            INC     DI
0780:0073 E2E5          LOOP    005A
0780:0075 B409          MOV     AH,09
0780:0077 8D161000      LEA     DX,[0010]
0780:007B CD21          INT     21
0780:007D B8004C        MOV     AX,4C00
0780:0080 CD21          INT     21
```

2 ID NUM

```
-d 0 b
077E:0000  32 32 33 36 31 31 34 39-36 38 00 00                  2236114968..
```

3 dos NUM ASCII 4

```
-g 007d
5
6
Q01DCh
```

4 3 ID NUM

```
-d 0 b
077E:0000  32 32 33 36 31 31 34 39-36 38 DC 01                  2236114968..
```

5

```
; empty asm file

title I love asm

data segment
    ID          db    '2 236 114 968'
    num         dw    0
    old_off dw    ?                    ;
    old_seg dw    ?                    ;
    hex_str db    4 dup(?), 'h$'       ;
data ends

code segment
    assume cs:code, ds:data

    count    proc   far
        push   ds
        push   ax
        mov    ax, data
        mov    ds, ax
        inc    num
```

```
    pop     ax
    pop     ds
    iret
count   endp

main    proc
    ; assign the data segment base address to DS
    mov     ax, data
    mov     ds, ax

    ; TODO ...
    ; | add your code between arrows |
    ; v ---------------------------- v
    mov     ah, 35h
    mov     al, 1Ch
    int     21h
    push    es
    pop     ax
    mov     old_seg, ax
    mov     old_off, bx

    ;       1Ch         count
    push    cs
    pop     ax
    mov     dx, offset count
    mov     ds, ax
    mov     ah, 25h
    mov     al, 1Ch
    int     21h

    ;       DS
```

```
        mov     ax,  data
        mov     ds,  ax

wait_input:
        mov     ah,  01h
        int     21h
        cmp     al,  'Q'
        jne     wait_input

        ;       1CH
        mov     dx,  old_off
        mov     ax,  old_seg
        mov     ds,  ax
        mov     ah,  25h
        mov     al,  1Ch
        int     21h
        mov     ax,  data
        mov     ds,  ax

        ;   NUM
        mov     ax,  num
        mov     cx,  4
        lea     di,  hex_str
convert_loop:
        push    cx
        mov     cl,  4
        rol     ax,  cl
        pop     cx
        mov     bl,  al
        and     bl,  0Fh                ;     4
        add     bl,  30h                ;       ASCII
```

```
        cmp     bl, 39h
        jbe     is_digit                ; 0-9
        add     bl, 7                   ; A-F   7
is_digit:
        mov     byte ptr [di], bl
        inc     di
        loop    convert_loop

        ;
        mov     ah, 09h
        lea     dx, hex_str
        int     21h
        ; ^ ---------------------------- ^
        ; |           The END            |

        ; method 2: return to dos
        mov     ax, 4c00h
        int     21h
    main    endp
code ends
end main
```

3 BIOS DOS

1

```
-u
0780:0000 B87E07        MOV     AX,077E
0780:0003 8ED8          MOV     DS,AX
0780:0005 8D1E1400      LEA     BX,[0014]
0780:0009 B90A00        MOV     CX,000A
0780:000C 8D3E0A00      LEA     DI,[000A]
0780:0010 B401          MOV     AH,01
0780:0012 CD21          INT     21
0780:0014 3C0D          CMP     AL,0D
0780:0016 7411          JZ      0029
0780:0018 3C30          CMP     AL,30
0780:001A 72F4          JB      0010
0780:001C 3C39          CMP     AL,39
0780:001E 77F0          JA      0010
```

```
0780:0020 2C30          SUB     AL,30
0780:0022 D7            XLAT
0780:0023 8805          MOV     [DI],AL
0780:0025 47            INC     DI
0780:0026 49            DEC     CX
0780:0027 75E7          JNZ     0010
0780:0029 B8004C        MOV     AX,4C00
0780:002C CD21          INT     21
```

2 ID BUFFER

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 00 00 00 00 00 00   2236114968......
077E:0010  00 00 00 00                                       ....
```

3 ID BUFFER

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 07 05 09 01 03 06   2236114968......
077E:0010  08 00 02 04                                       ....
```

4

; empty asm file

title I love asm

```
data  segment
      ID          db    '2 236 114 968'
      buffer      db    10  dup(?)
      table       db    7,5,9,1,3,6,8,0,2,4
data  ends

code  segment
      assume  cs:code,  ds:data
      main        proc
            ; assign  the  data  segment  base  address  to  DS
            mov      ax,  data
            mov      ds,  ax

            ;  TODO  ...
           ; |  add  your  code  between  arrows  |
           ;  v  ---------------------------  v
            lea      bx,  table
           mov      cx,  10
           lea      di,  buffer

input_loop:
           mov      ah,  01h
           int      21h

           ;                0Dh
           cmp      al,  0Dh
           je       end_input                    ;

           ;
           cmp      al,  '0'
```

```
        jb      input_loop
        cmp     al, '9'
        ja      input_loop

        sub     al, '0'                 ;   ASCII           0-9

        xlat                            ;           AL

        mov     [di], al
        inc     di                          ;

        dec     cx
        jnz     input_loop

end_input:
        ; ^ ---------------------------  ^
        ; |               The END                 |

        ; method 2: return to dos
        mov     ax, 4c00h
        int     21h
    main    endp
code ends
end main
```